Reporte · Actividad 5: Huffman y árbol de expansión mínima

Nov 9, 2026 · @SalvadorRG

Campo	Dato
Materia	Análisis de Algoritmos (2026B)
Actividad	5 · Algoritmo de Huffman y árbol de expansión mínima
Modalidad	Individual
Alumno	Salvador (@SalvadorRGDev)
Repositorio	Analisis-de-algoritmos/act5_huffman
Lenguaje	Python 3.14 con Tkinter y matplotlib

Por completar: nombre completo, institución y profesor.

1. Introducción

Se implementó un compresor de texto basado en el algoritmo de Huffman que reduce un libro real de Project Gutenberg en 45.48 % y lo recupera sin perder un solo byte. La aplicación tiene interfaz gráfica y guarda los resultados en un formato propio, `.hada`.

Un archivo de texto suele usar 8 bits o más por carácter, aunque unas cuantas letras aparecen mucho más que otras. Huffman (1952) aprovecha esa diferencia: asigna códigos binarios cortos a los símbolos frecuentes y largos a los raros, sin que ningún código sea prefijo de otro. Así el texto se puede leer de nuevo sin separadores.

El algoritmo es **voraz**. En cada paso toma la mejor opción local, que es unir los dos nodos de menor frecuencia, y nunca la revisa. Esa elección produce un código prefijo óptimo (Cormen et al., 2022). Para comparar con otro problema que se resuelve con la misma técnica, se implementó también el **árbol de expansión mínima** con los algoritmos de Kruskal (1956) y Prim (1957).

Objetivo. Implementar y analizar el algoritmo de Huffman para comprimir textos mediante un árbol binario construido a partir de la frecuencia de cada carácter.

Alcance. La aplicación:

1. Lee un `.txt` local o un libro buscado en Project Gutenberg.
2. Calcula las frecuencias, construye el árbol y genera los códigos.
3. Codifica el texto y lo guarda en un archivo `.hada`.
4. Inspecciona ese archivo sin el original, lo decodifica y verifica que el resultado sea idéntico.
5. Compara el tamaño original con el comprimido.
6. Resuelve el árbol de expansión mínima con Kruskal y con Prim.

2. Desarrollo

2.1 Algoritmo de Huffman

El compresor sigue cinco pasos, implementados en `huffman.py` sin bibliotecas de compresión. Con n caracteres, k símbolos distintos y B bits de salida, el costo total es O(n + k log k + B).

Paso	Función	Qué hace	Complejidad
1. Frecuencias	`calcular_frecuencias`	Cuenta cada carácter en un solo recorrido del texto.	O(n)
2. Árbol voraz	`construir_arbol`	Mete cada símbolo como hoja en un min-heap (Python Software Foundation, s. f.-a) y une los dos de menor frecuencia hasta que queda uno.	O(k log k)
3. Códigos	`generar_codigos`	Recorre el árbol: rama izquierda = 0, derecha = 1.	O(k · profundidad)
4. Codificar	`codificar`	Concatena los códigos y empaqueta los bits de 8 en 8, rellenando el último byte con ceros.	O(n + B)

Paso	Función	Qué hace	Complejidad
5. Decodificar	`decodificar`	Recorre el árbol bit por bit; al llegar a una hoja emite su símbolo y vuelve a la raíz.	O(B)

El núcleo voraz son las k − 1 uniones del paso 2:

```
while len(cola) > 1:
    f1, _, menor = heapq.heappop(cola)      # menor frecuencia
    f2, _, segundo = heapq.heappop(cola)    # segunda menor
    nuevo = Nodo(f1 + f2, izquierdo=menor, derecho=segundo)
    heapq.heappush(cola, (nuevo.frecuencia, orden, nuevo))
    orden += 1
```

El contador `orden` desempata nodos con la misma frecuencia por orden de llegada. Además, las hojas entran a la cola ordenadas por (frecuencia, código Unicode). Con eso, el mismo texto produce siempre el mismo árbol, y el descompresor puede reconstruirlo solo con la tabla de frecuencias.

Por qué la elección voraz es óptima. Los dos símbolos menos frecuentes pueden ir como hermanos en el nivel más profundo de un árbol óptimo, porque intercambiarlos con otros nodos no aumenta el costo. Al unirlos queda un problema con k − 1 símbolos que tiene la misma estructura (subestructura óptima). Por eso la longitud media L de Huffman queda entre la entropía H (Shannon, 1948) y H + 1:

$$H = -\sum p_i \cdot \log_2 p_i \quad \leqslant \quad L = \sum p_i \cdot \ell_i \quad < \quad H + 1$$

Aquí p_i es la proporción del símbolo i y ℓ_i la longitud de su código.

Símbolos. Se trabaja con caracteres, no con bytes. El archivo se decodifica como UTF-8 y, si no es válido, como Latin-1. Volver a codificar el texto da exactamente los mismos bytes en ambos casos. Así, letras como ñ o á cuentan como un solo símbolo.

2.2 Formato propio `.hada`

El archivo `.hada` (*Huffman · Análisis De Algoritmos*) guarda todo lo necesario para recuperar el texto sin el `.txt` original, pero no el texto en sí. Todos los enteros van en big-endian. La especificación completa está en `docs/FORMATO.md` del repositorio.

Offset	Bytes	Campo	Para qué sirve
0	4	Firma `HADA`	Identifica el formato; si falta, el archivo se rechaza.
4	1	Versión	Hoy es 1.
5	1	Codificación	0 = UTF-8, 1 = Latin-1.
6	2	Reservado	Espacio para versiones futuras.
8	8	Tamaño original	Bytes del `.txt`, para calcular el ahorro.
16	8	Caracteres	Debe coincidir con la suma de la tabla.
24	8	Bits válidos	Dice dónde termina el último código.
32	1	Bits de relleno	0 a 7 ceros del último byte, que se ignoran.
33	32	SHA-256	Hash del original, para verificar la integridad.
65	8	Fecha	Momento de la compresión (segundos Unix).
73	2 + n	Nombre	Nombre del `.txt` original en UTF-8.
75 + n	4 + 8k	Tabla	k pares (código Unicode, frecuencia).
79 + n + 8k	resto	Datos	Bits de Huffman empaquetados.

Se guardan las **frecuencias** y no el árbol. Con ellas, el descompresor vuelve a ejecutar el mismo algoritmo voraz y obtiene el mismo árbol, gracias al orden de desempate de la sección 2.1. Además, el inspector puede mostrar la tabla real del texto.

Los **bits válidos** resuelven el problema del último byte. Por ejemplo, en `abracadabra` los datos ocupan 23 bits, y el byte final `11011100` termina con un 0 que es relleno. Sin ese dato, el descompresor leería ese cero como otra letra `a`.

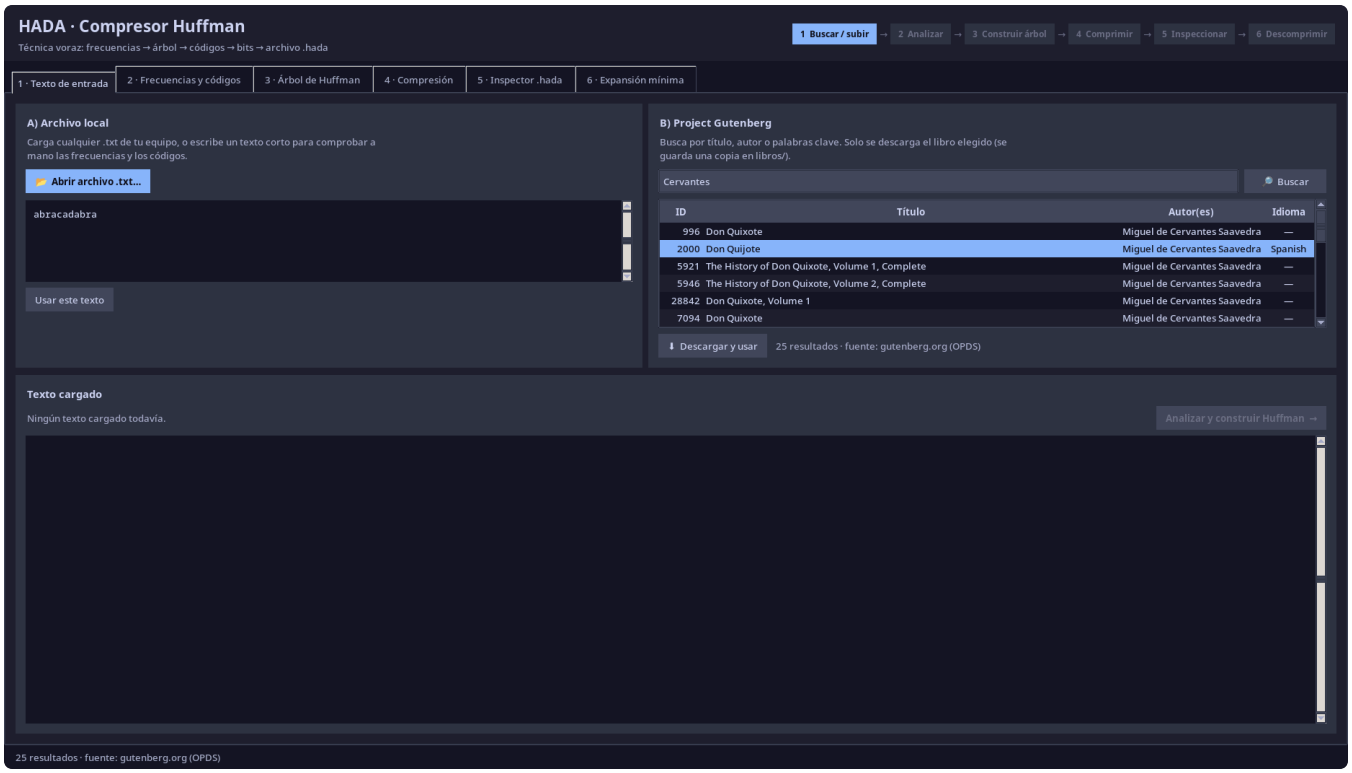
Para el Quijote, la cabecera y la tabla pesan 977 bytes, el 0.08 % del archivo comprimido.

2.3 Interfaz gráfica

La interfaz (Tkinter y matplotlib; Python Software Foundation, s. f.-b) tiene una pestaña por cada etapa del flujo, y una barra superior marca en verde las etapas terminadas: **buscar o subir → analizar → construir árbol → comprimir → inspeccionar → descomprimir**. Las tareas largas corren en un hilo aparte para que la ventana no se congele.

Entrada. Se puede abrir un `.txt` local, escribir un texto corto o buscar en Project Gutenberg por título, autor o palabra clave. La búsqueda usa el catálogo OPDS de gutenberg.org y, si no responde, Gutendex. Solo se descarga el libro elegido.

Figura 1. *Búsqueda «Cervantes» en Project Gutenberg; se elige el libro #2000, Don Quijote.*



Frecuencias y códigos. La tabla se puede ordenar por cualquier columna. La gráfica muestra que los símbolos más frecuentes reciben los códigos más cortos. Las tarjetas comparan la longitud media (4.4741 bits) con la entropía (4.4378 bits).

Figura 2. *Frecuencias del Quijote: 111 símbolos; el espacio aparece 357,678 veces y recibe un código de 3 bits.*



Árbol. El árbol tiene zoom y desplazamiento. Al hacer clic en una hoja se marca su camino desde la raíz y aparece su código. La construcción se puede reproducir paso a paso: en cada unión se resaltan los dos nodos elegidos, y los nodos que todavía no existen se ven como fantasmas.

Figura 3. *Árbol completo del Quijote con el camino de la letra «e» (código 010) resaltado en verde.*

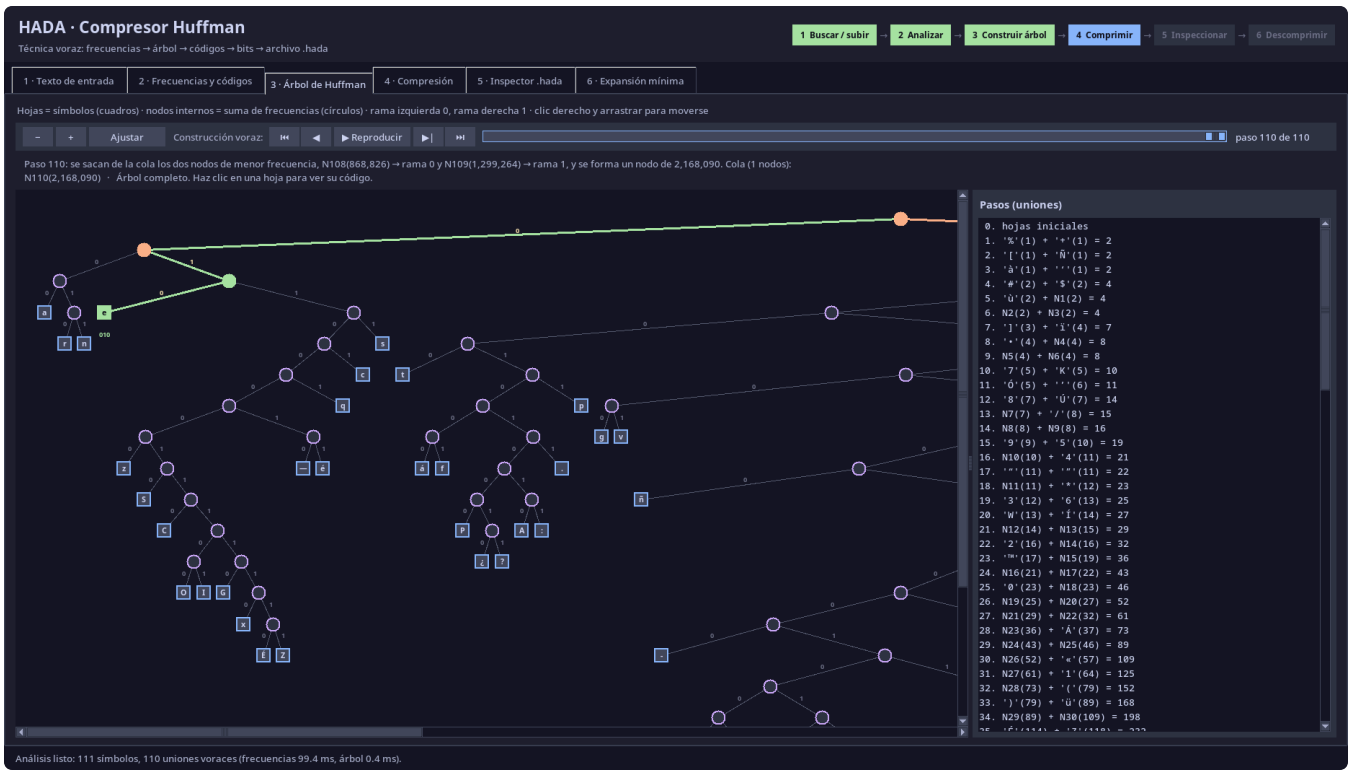
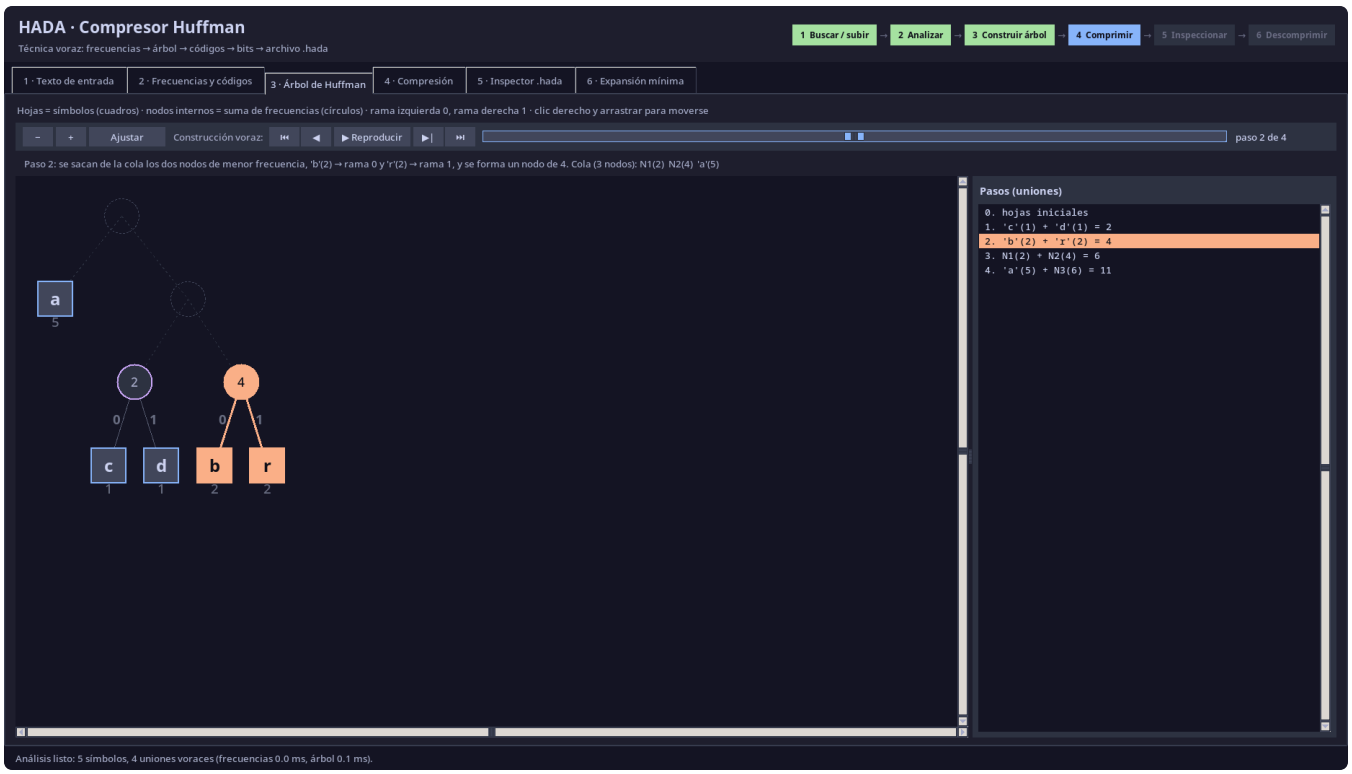
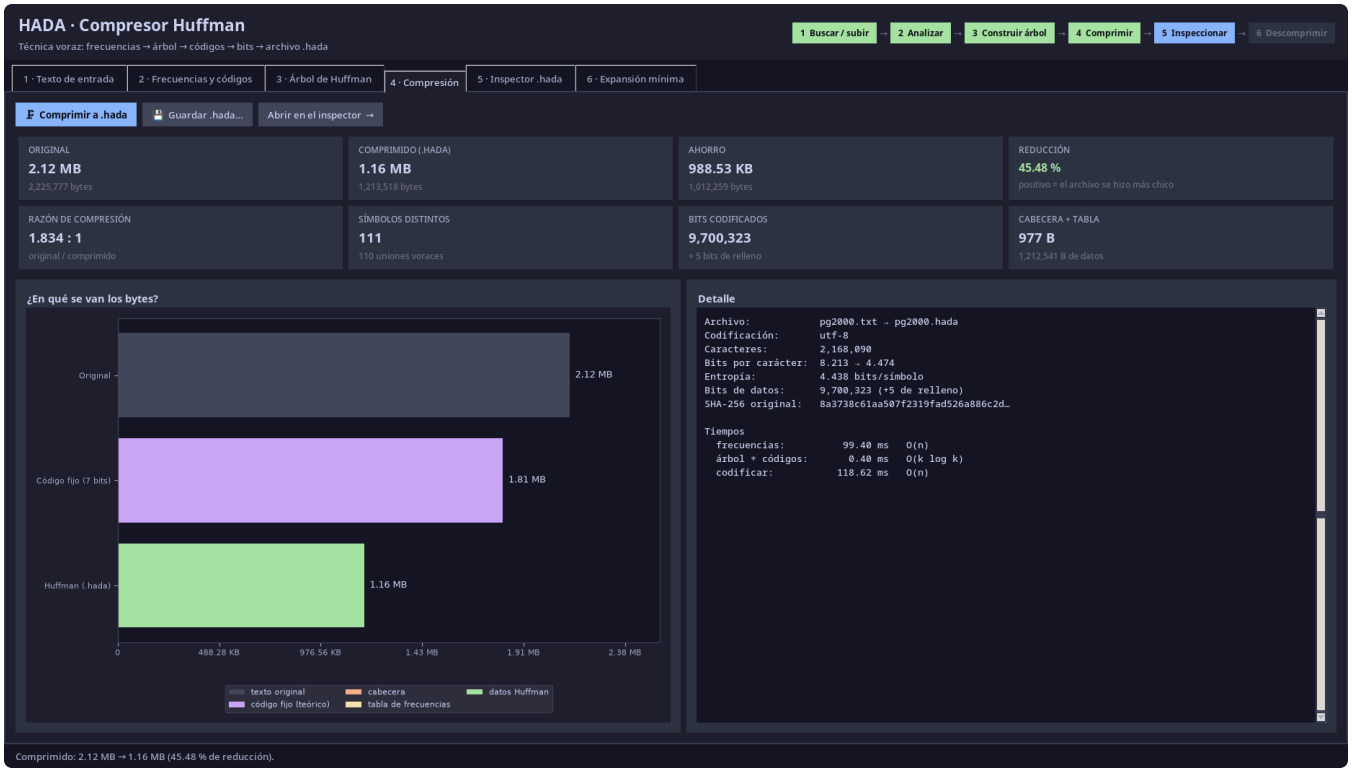


Figura 4. *Paso 2 de 4 con «abracadabra»: se unen b(2) y r(2) en un nodo de 4. Debajo se muestra la cola de prioridad en ese momento.*



Compresión. Muestra el tamaño original y el comprimido en bytes y en KB/MB, el ahorro, el porcentaje de reducción, la razón de compresión, los símbolos, los bits y los tiempos. La gráfica compara el original, un código de longitud fija y Huffman.

Figura 5. *Compresión del Quijote: de 2.12 MB a 1.16 MB, 45.48 % de reducción y razón 1.834 : 1.*



Inspector. Abre solo el `.hada` y muestra cada campo con su offset y su tamaño, la tabla y los códigos reconstruidos, el árbol y un volcado hexadecimal coloreado por sección. También separa los primeros bits por código y marca en rojo los bits de relleno. La descompresión se ejecuta después, con un botón aparte.

Figura 6. *Estructura del archivo pg2000.hada: 15 campos y la sección de datos, que empieza en el byte 977.*

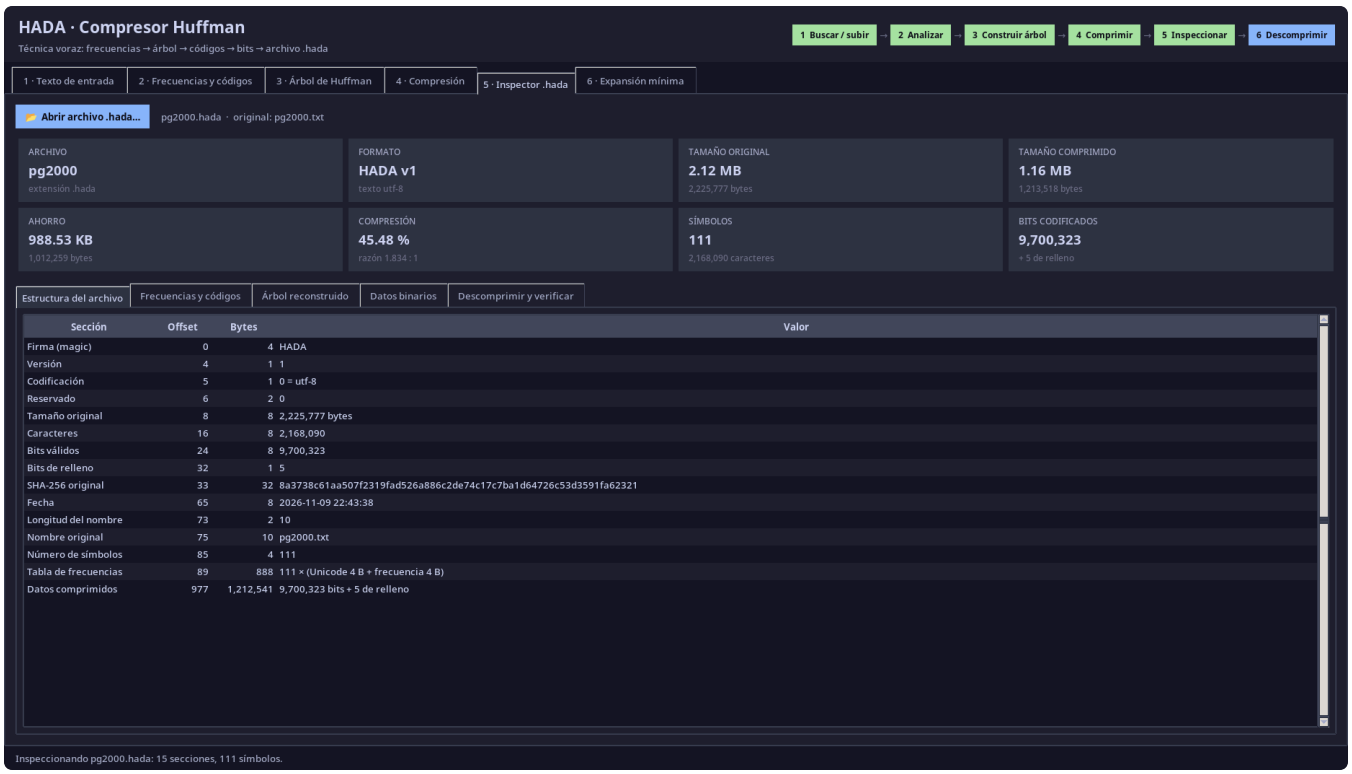


Figura 7. *Datos binarios de abracadabra.hada: volcado hexadecimal por sección, los 23 bits separados por código y el bit de relleno en rojo.*

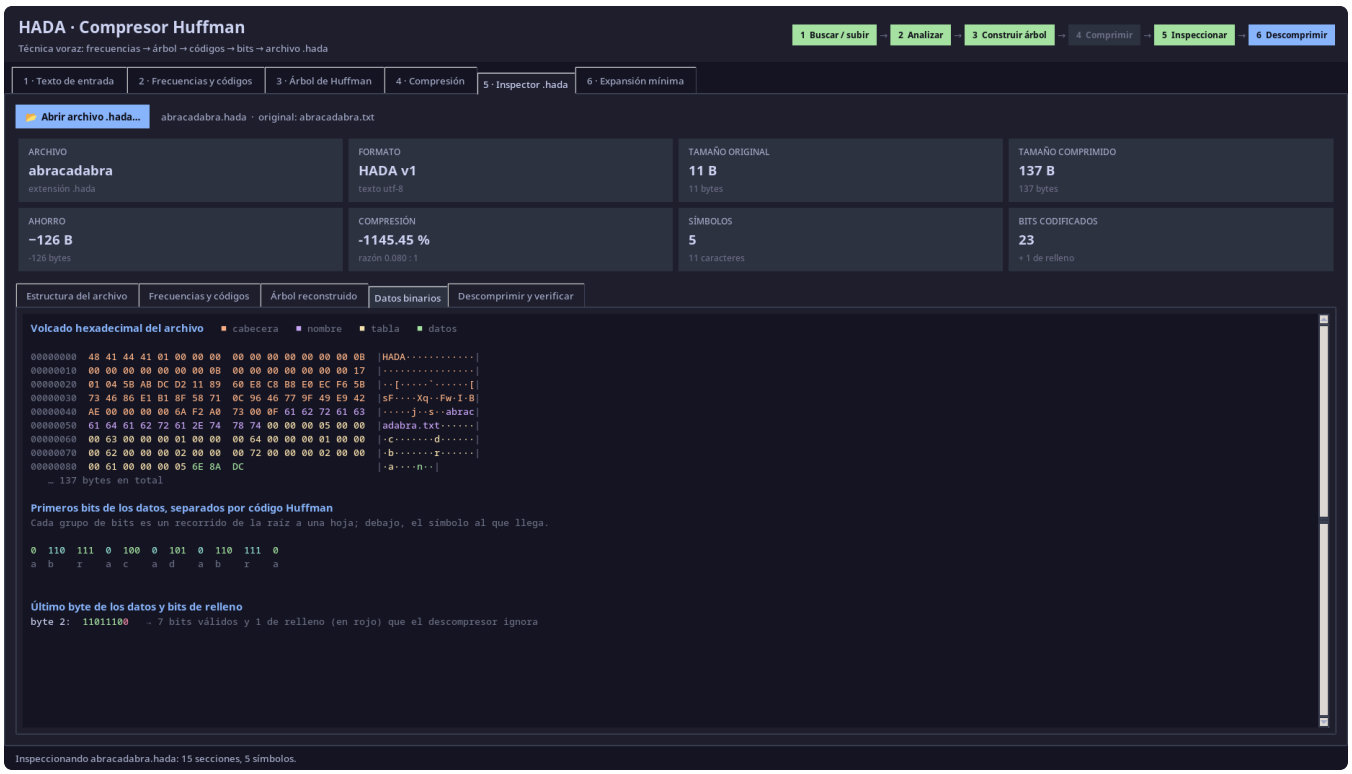
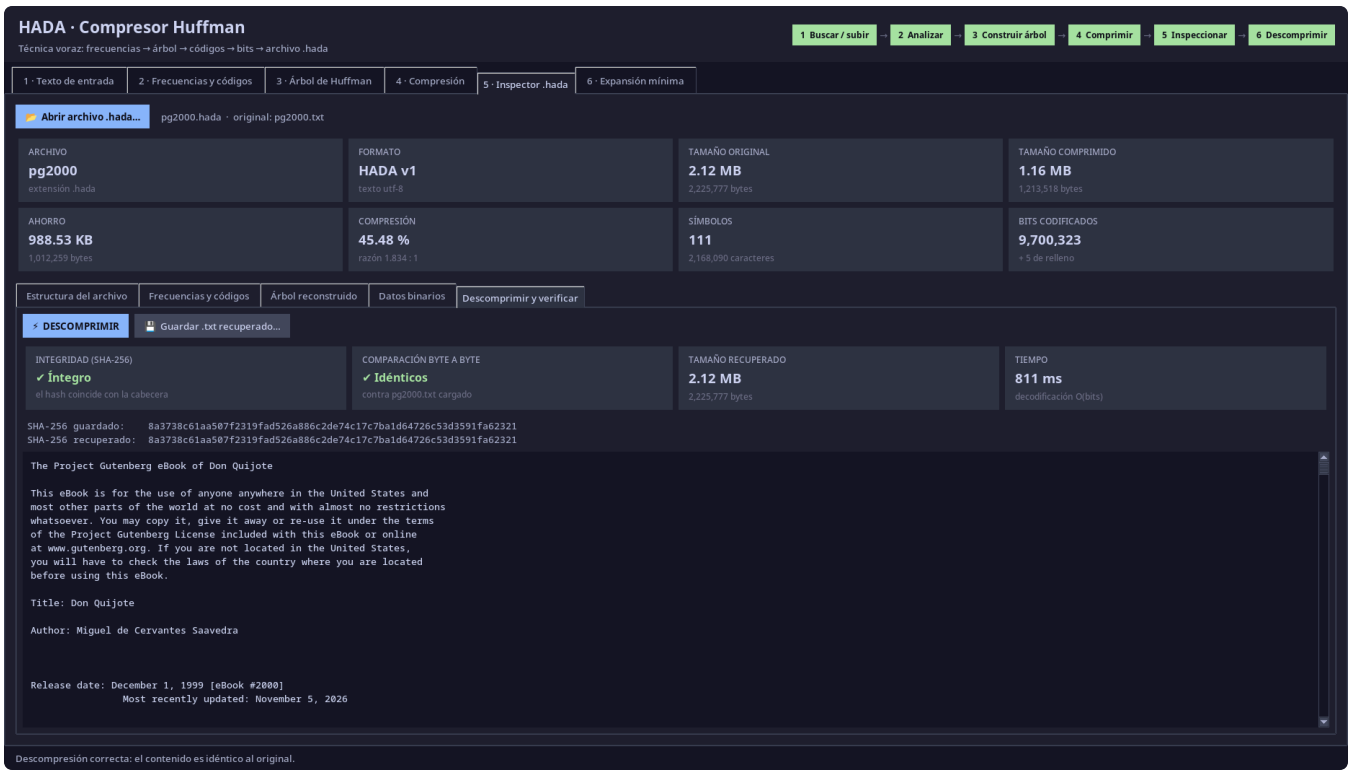


Figura 8. *Descompresión del Quijote: el SHA-256 coincide y la comparación byte a byte con el original da «idénticos».*



2.4 Árbol de expansión mínima

Kruskal y Prim llegan al mismo peso mínimo, 37, en el grafo clásico de Cormen et al. (2022), y también coinciden en 100 grafos aleatorios. Están en `expansion_minima.py`. Igual que Huffman, ambos son voraces: en cada paso toman la arista más barata que está permitida.

Algoritmo	Elección voraz en cada paso	Estructura de apoyo	Complejidad
Kruskal (1956)	La arista más ligera de todo el grafo que no forme un ciclo.	Union-Find con compresión de camino y unión por rango.	O(E log E)
Prim (1957)	La arista más ligera que sale del árbol que se va formando.	Min-heap de aristas candidatas.	O(E log V)

Los dos se apoyan en la **propiedad del corte**: si se divide el grafo en dos partes, la arista más ligera que las conecta pertenece a algún árbol de expansión mínima. Es el mismo tipo de argumento que justifica a Huffman.

En la pestaña «Expansión mínima» se carga el grafo de ejemplo o uno aleatorio, que se puede editar como texto (`origen destino peso`). El recorrido avanza paso a paso: las aristas aceptadas se ven en verde, la del paso actual en naranja y las que formarían un

ciclo en rojo. Al final se compara el peso que obtiene el otro algoritmo.

Figura 9. *Kruskal en el paso 5: acepta C–F (4) y lleva un peso acumulado de 13. En la lista, los pasos 6, 8 y 10 se rechazan porque formarían un ciclo.*

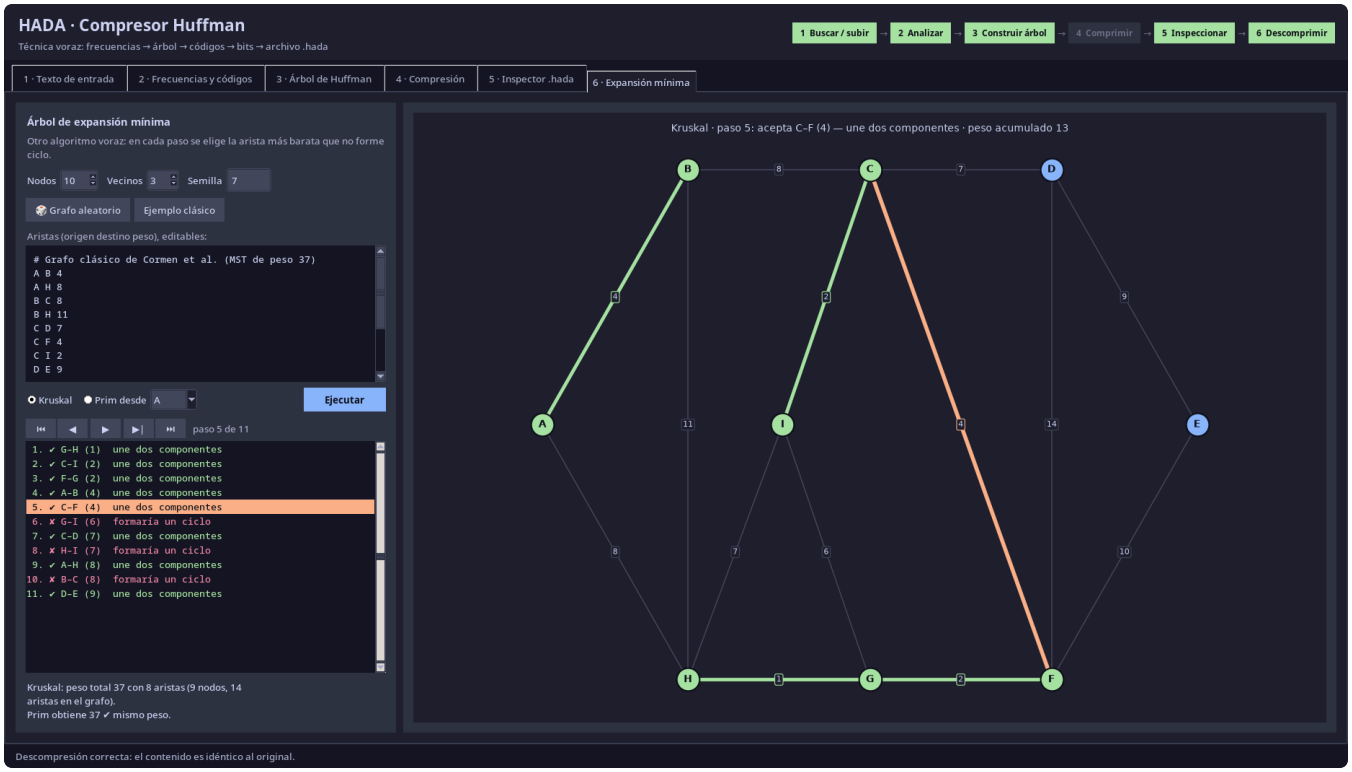
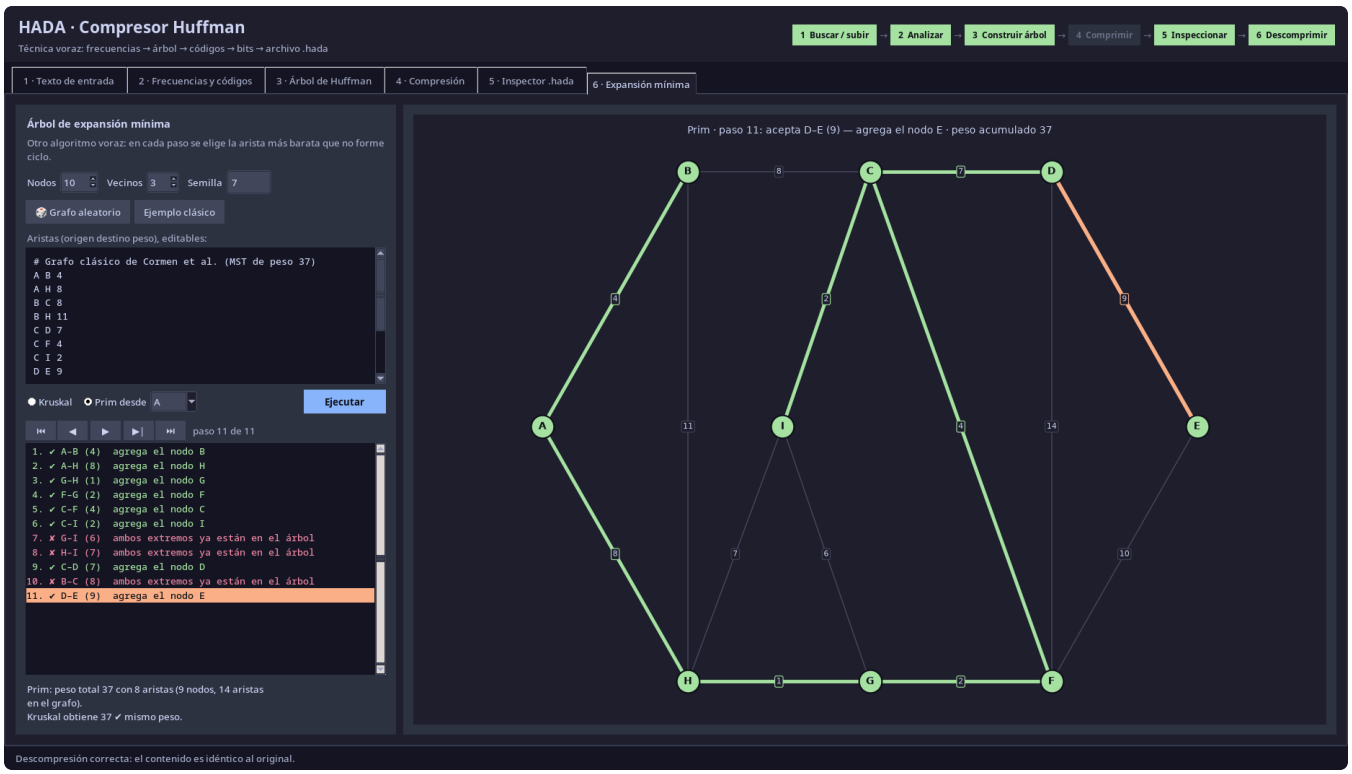


Figura 10. *Prim desde el nodo A en el mismo grafo: peso total 37 con 8 aristas, igual que Kruskal.*



2.5 Pruebas y resultados

Los tres libros (Cervantes Saavedra, 1605/1999; Pardo Bazán, 1886/2006; Shelley, 1818/1993) se redujeron entre 43.81 % y 45.48 %, y en todos los casos el archivo recuperado fue idéntico al original. Las 32 pruebas unitarias de `tests/test_huffman.py` pasan. Los resultados se generaron con `evidencias/generar_evidencias.py`.

Caso	Original	`.hada`	Reducción	Razón	Símbolos	Bits/símbolo	Entropía	Integridad
Texto pequeño `abracadabra`	11 B	137 B	−1145.45 %	0.080 : 1	5	2.091	2.040	Idéntico
Don Quijote (#2000, español)	2,225,777 B (2.12 MB)	1,213,518 B (1.16 MB)	45.48 %	1.834 : 1	111	4.474	4.438	Idéntico
Frankenstein (#84, inglés)	448,885 B (438.36 KB)	252,247 B (246.33 KB)	43.81 %	1.780 : 1	94	4.504	4.469	Idéntico
Los pazos de Ulloa (#18005, no usado al programar)	535,176 B (522.63 KB)	299,519 B (292.50 KB)	44.03 %	1.787 : 1	105	4.571	4.537	Idéntico

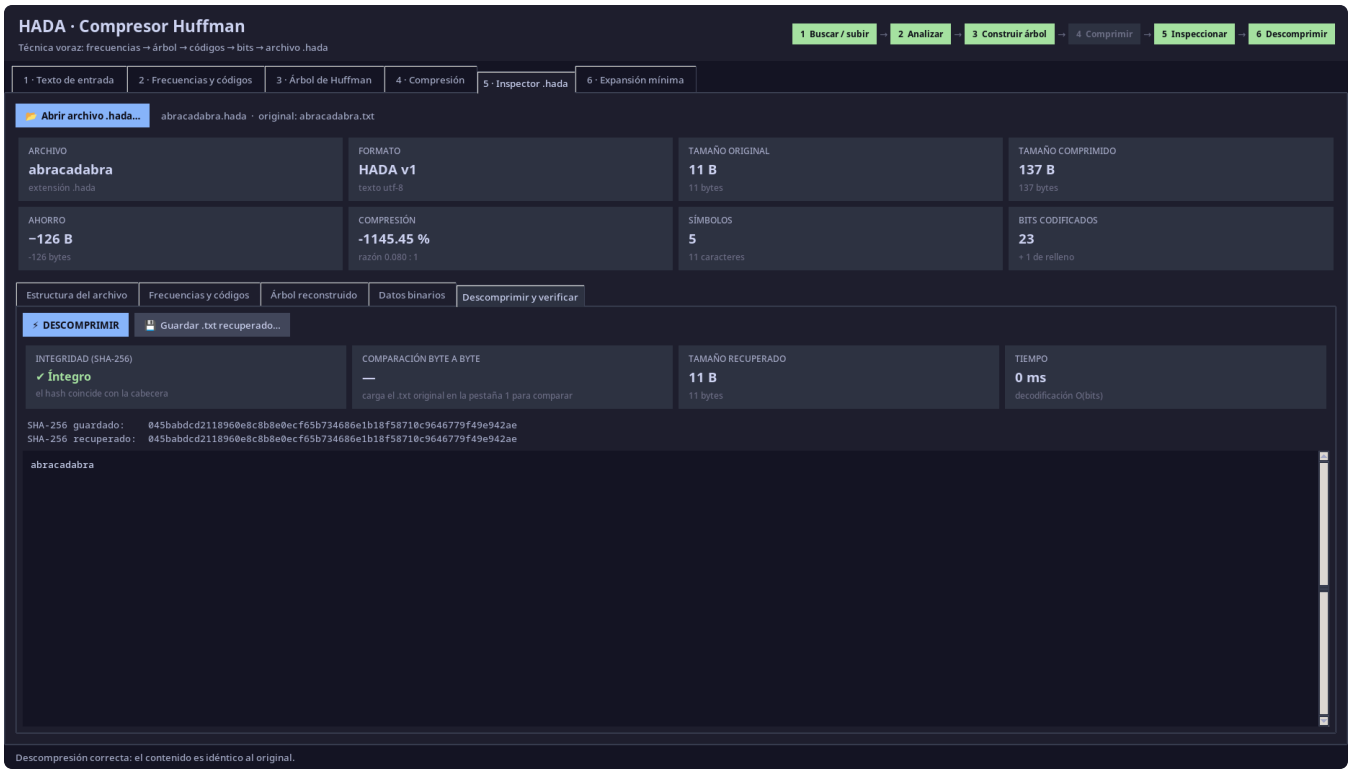
Las cuatro pruebas mínimas se cubrieron así:

1. **Texto pequeño.** En `abracadabra` las frecuencias son a:5, b:2, r:2, c:1 y d:1. Las uniones voraces son 1+1=2, 2+2=4, 2+4=6 y 5+6=11, y producen los códigos a=0, c=100, d=101, b=110 y r=111. El texto ocupa 5·1 + 6·3 = 23 bits, igual a lo que muestra el inspector (Figura 7). Una prueba unitaria revisa estos valores bit por bit.
2. **Libro de Project Gutenberg.** El Quijote, con 2,168,090 caracteres, se comprime en 0.23 s y se descomprime en 0.69 s (Figuras 5 y 8).
3. **Integridad.** Se compara el SHA-256 guardado en la cabecera con el del resultado y, si el original está cargado, también byte a byte. Las pruebas alteran un bit de los datos y una frecuencia de la tabla, y en ambos casos se detecta el error.
4. **Archivo no usado al programar.** *Los pazos de Ulloa* se descargó por primera vez para esta prueba. El script acepta cualquier otro ID de Gutenberg o `.txt`, para la prueba en vivo.

Otras pruebas cubren un archivo vacío, un solo símbolo, emojis y caracteres chinos, saltos de línea `\r\n`, BOM, Latin-1 y bytes aleatorios. También hay comparaciones contra fuerza bruta: Huffman da el menor número de bits posible en alfabetos pequeños, y Kruskal da el árbol de menor peso.

En los tres libros, Huffman queda a menos de 0.04 bits por símbolo de la entropía. Un código de longitud fija necesitaría 7 bits para 111 símbolos. En cambio, el texto pequeño crece, porque la cabecera y la tabla (137 − 3 = 134 bytes) pesan más que el ahorro.

Figura 11. *Inspector con solo abracadabra.hada, sin el .txt original: la descompresión recupera «abracadabra» y el SHA-256 coincide.*



3. Conclusiones

La elección voraz de Huffman, que consiste en unir siempre los dos nodos menos frecuentes, bastó para reducir libros reales alrededor de 44–45 %. El resultado quedó a menos de 0.04 bits por símbolo del límite teórico de Shannon. La teoría predice que un código prefijo óptimo queda entre H y H + 1, y los datos lo confirman.

La compresión solo es útil si se puede deshacer sin el original. Por eso el formato tuvo que guardar algo más que los bits: la tabla de frecuencias para reconstruir el mismo árbol, los bits válidos para ignorar el relleno y un hash para verificar la integridad. El desempate determinista en la cola de prioridad fue indispensable, porque sin él dos árboles distintos podrían salir de la misma tabla.

Huffman tiene límites claros. En textos muy cortos el archivo crece, porque la tabla pesa más que el ahorro. Además, al codificar carácter por carácter no aprovecha palabras o frases repetidas; para eso existen métodos de diccionario como LZ77.

Comparar Huffman con Kruskal y Prim muestra que la técnica voraz funciona cuando el problema cumple dos condiciones. Debe tener la propiedad de elección voraz (una decisión local segura) y subestructura óptima (lo que queda es el mismo problema, pero

más chico). En los tres algoritmos, una cola de prioridad o un ordenamiento hace que cada decisión cueste O(log n).

La interfaz ayudó a entender el algoritmo y no solo a ejecutarlo. Reproducir la construcción del árbol paso a paso y ver los bits separados por código en el inspector hace visible cómo las frecuencias se convierten en un árbol, el árbol en códigos y los códigos en bytes.

Referencias

Cervantes Saavedra, M. de. (1999). *Don Quijote* [Libro electrónico]. Project Gutenberg. https://www.gutenberg.org/ebooks/2000 (Obra original publicada en 1605 y 1615)

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE, 40*(9), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society, 7*(1), 48–50. https://doi.org/10.1090/S0002-9939-1956-0078686-7

Pardo Bazán, E. (2006). *Los pazos de Ulloa* [Libro electrónico]. Project Gutenberg. https://www.gutenberg.org/ebooks/18005 (Obra original publicada en 1886)

Prim, R. C. (1957). Shortest connection networks and some generalizations. *Bell System Technical Journal, 36*(6), 1389–1401. https://doi.org/10.1002/j.1538-7305.1957.tb01515.x

Python Software Foundation. (s. f.-a). *heapq — Heap queue algorithm*. Python 3 documentation. Recuperado el 9 de noviembre de 2026, de https://docs.python.org/3/library/heapq.html

Python Software Foundation. (s. f.-b). *tkinter — Python interface to Tcl/Tk*. Python 3 documentation. Recuperado el 9 de noviembre de 2026, de https://docs.python.org/3/library/tkinter.html

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal, 27*(3), 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

Shelley, M. W. (1993). *Frankenstein; or, the modern Prometheus* [Libro electrónico]. Project Gutenberg. https://www.gutenberg.org/ebooks/84 (Obra original publicada en 1818)